

Programação Web

Spring Boot e Início do Projeto (RestControllers, DTOs e Organização do projeto)

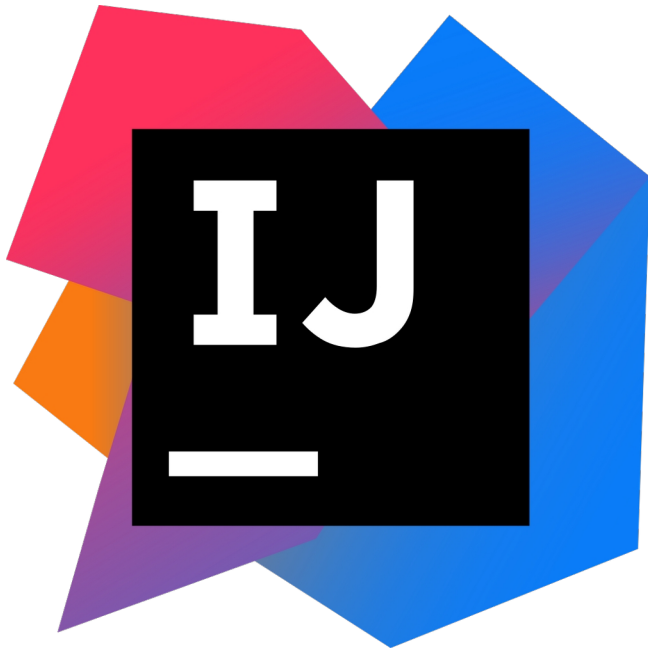
Prof. Dr. Gabriel Gadelha

Aula 05

Objetivos

- Ferramentas
- Criação do primeiro projeto
- Anotação @Component e @Value
- Criação do primeiro endpoint
- Criação de DTOs
- Organização do projeto

Ferramentas



Insomnia

start.spring.io



Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Java ☐ Kotlin ☐ Groovy
☒ Maven

Language

Spring Boot

☐ 4.2.0 (SNAPSHOT) ☐ 4.2.0 (M1) ☐ 4.1.2 (SNAPSHOT) ☒ 4.1.1
☐ 4.0.9 (SNAPSHOT) ☐ 4.0.8

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☒ 21 ☐ 17

Dependencies

[ADD DEPENDENCIES...](#) CTRL + B

Spring Web WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

Spring Boot DevTools DEVELOPER TOOLS

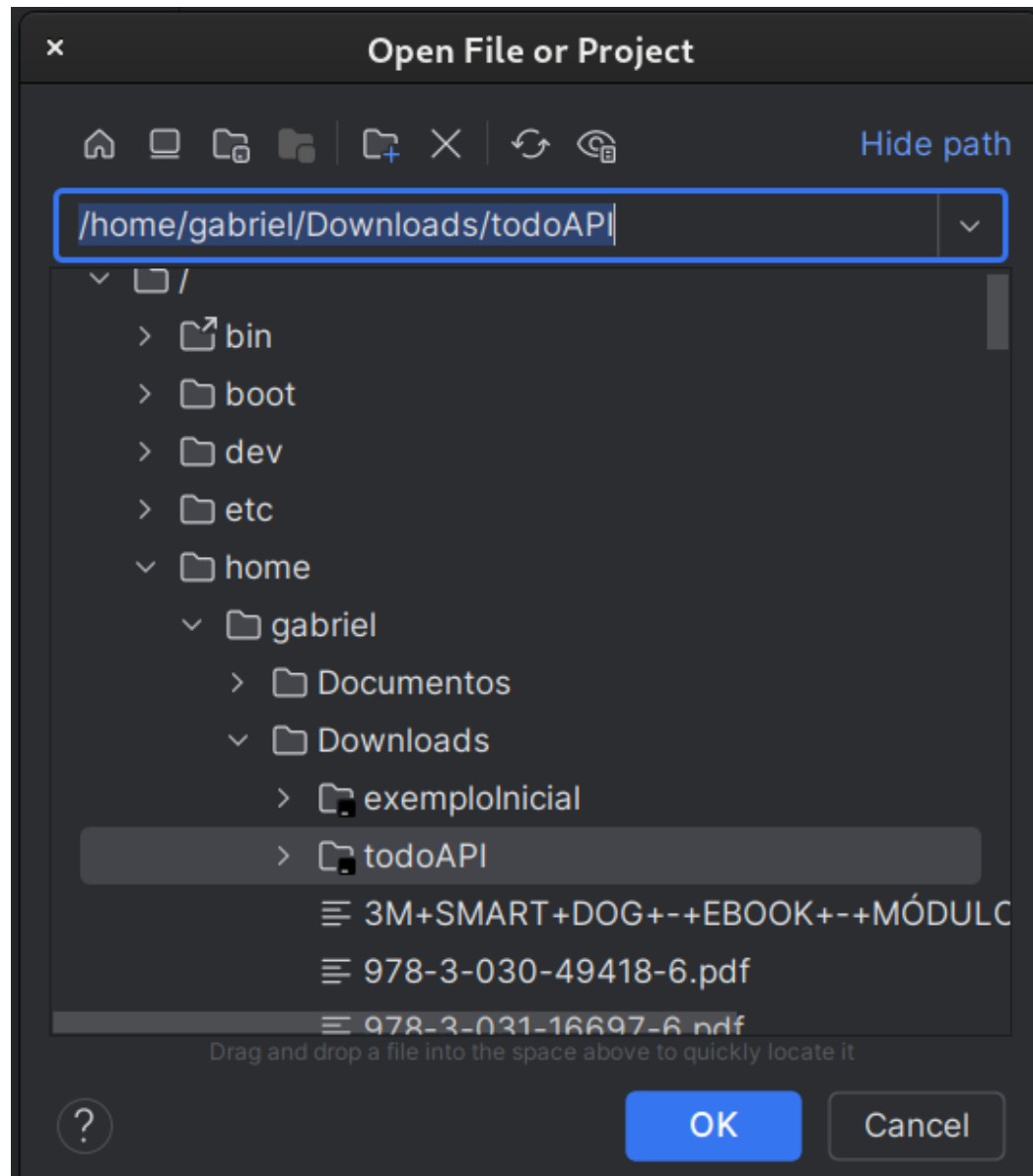
Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

[GENERATE](#) CTRL + ⌘

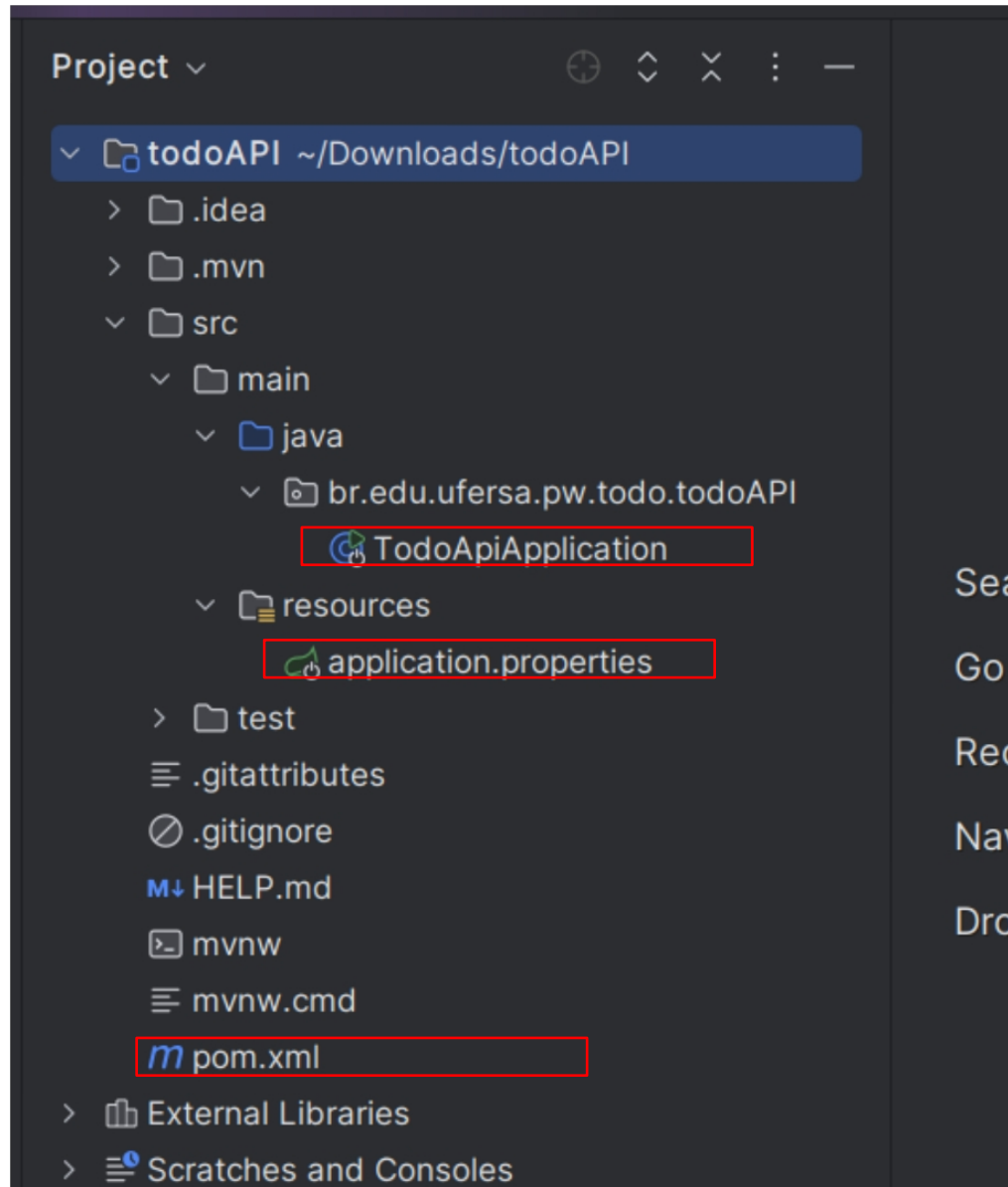
[EXPLORE](#) CTRL + SPACE

...

Abrir o projeto



Arquivos Principais



Iniciar o Projeto

Project ▾

- ▾ todoAPI ~/Downloads/todoAPI
 - > .idea
 - > .mvn
 - ▾ src
 - ▾ main
 - ▾ java
 - ▾ br.edu.ufersa.pw.todo.todoAPI
 - TodoApiApplication
 - > resources
 - > test
 - .gitattributes
 - .gitignore
 - HELP.md
 - mvnw
 - mvnw.cmd
 - pom.xml
 - > External Libraries
 - > Scratches and Consoles

TodoApiApplication.java ×

```
1 package br.edu.ufersa.pw.to
2
3 > import ...
4
5
6  @SpringBootApplication
7
8  org.springframework.boot.autoconfigure
9
10 @Target({ElementType.TYPE})
11 @Retention(RetentionPolicy.RUNTIME)
12 @Documented
13 @Inherited
14 @SpringBootConfiguration
15 @EnableAutoConfiguration
16 @ComponentScan(excludeFilters = {@org.springframework
17 springframework.context.annotation.FilterType.CUST
18 TypeExcludeFilter.class}),@org.springframework.con
19 springframework.context.annotation.FilterType.CUST
20 AutoConfigurationExcludeFilter.class}))
21 public @interface SpringBootApplication
22 extends java.lang.annotation.Annotation
```

@Component

```
@SpringBootApplication
public class TodoApiApplication {
    public static void main(String[] args){
        SpringApplication.run(TodoApiApplication.class, args);}
    @Component
    public static class Runner implements ApplicationRunner {
        public void run(ApplicationArguments args) throws Exception{
            System.out.println("Rodou corretamente!");
        }
    }
}
```

Torna uma classe visível para a varredura automática do Spring Boot

@Value

```
@SpringBootApplication
public class TodoApiApplication {
    public static void main(String[] args){
        SpringApplication.run(TodoApiApplication.class, args);}
    @Component
    public static class Runner implements ApplicationRunner {
        @Value("${var1:valor default na classe TodoApiApplication}")
        String teste;
        public void run(ApplicationArguments args) throws Exception{
            System.out.println("Rodou corretamente!com"+teste);
        }
    }
}
```

Permite injetar valores de propriedades em campos de classe, construtores ou métodos

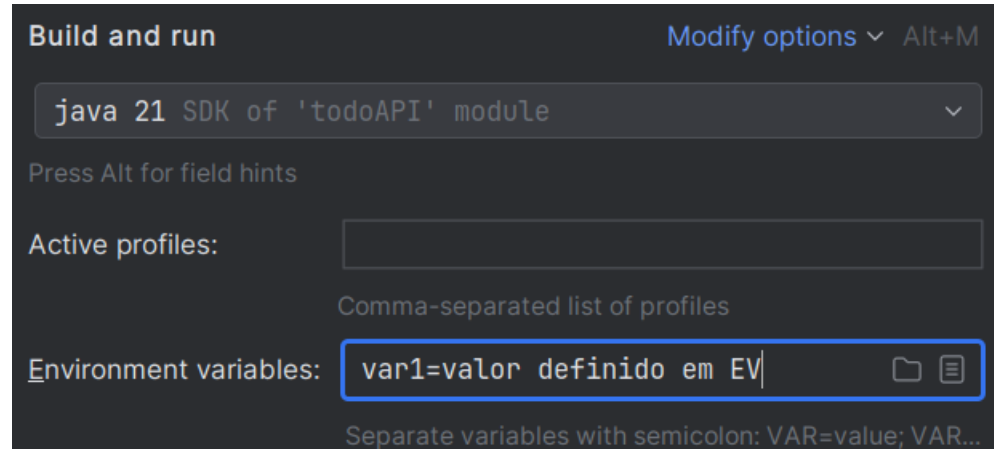
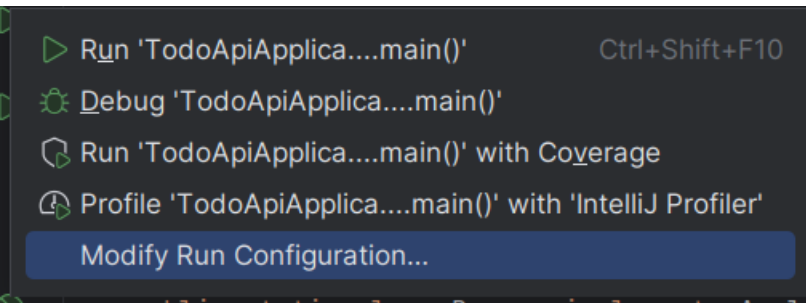
@Value

```
TodoApiApplication.java  application.properties x
1  spring.application.name=todoAPI
2  var1=valor definido no app.properties
3
```

```
2024-12-04T21:00:17.788-03:00 INFO 154494 --- [todoAPI] [
2024-12-04T21:00:17.934-03:00 INFO 154494 --- [todoAPI] [
2024-12-04T21:00:18.224-03:00 INFO 154494 --- [todoAPI] [
Rodou corretamente!! com valor definido no app.properties
```

Permite injetar valores de propriedades em campos de classe, construtores ou métodos

@Value



```
2024-12-04T21:02:32.303-03:00 INFO 154878 --- [todoAPI] [
Rodou corretamente!! com valor definido em EV
```

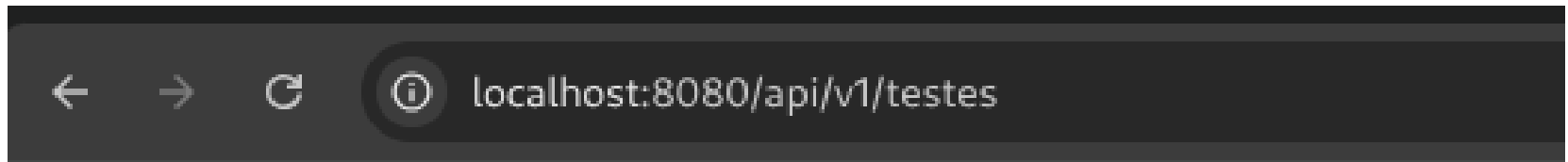
Permite injetar valores de propriedades em campos de classe, construtores ou métodos

Primeiro Endpoint

```
@RestController
@RequestMapping("/api/v1/testes")
public class TesteController {
    @GetMapping()
    public String testar(){
        return "Primeiro endpoint criado!!";}
}
```

- Define um controller com características REST
- Mapeia URLs para métodos específicos em controladores
- Define um método HTTP que lista todos os testes :)

Primeiro Endpoint



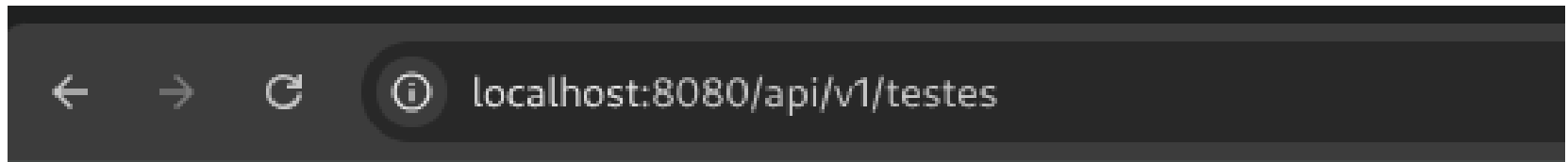
Primeiro endpoint criado!!

Primeiro Endpoint

```
@RestController
public class TesteController {
    @RequestMapping("/api/v1/testes")
    @GetMapping()
    public String testar() {
        return "Primeiro endpoint criado!!";
    }
}
```

- Define um controller com características REST
- Mapeia URLs para métodos específicos em controladores
- Define um método HTTP que lista todos os testes :)

Primeiro Endpoint



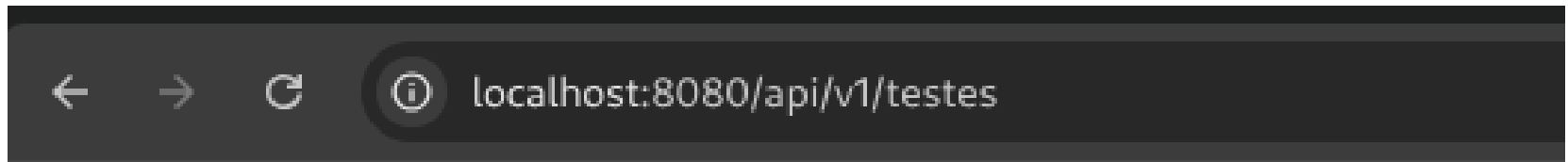
Primeiro endpoint criado!!

Primeiro Endpoint

```
@RestController
@RequestMapping("/api/v1")
public class TesteController {
    @GetMapping("/testes")
    public String testar() {
        return "Primeiro endpoint criado!!";
    }
}
```

- Define um controller com características REST
- Mapeia URLs para métodos específicos em controladores
- Define um método HTTP que lista todos os testes :)

Primeiro Endpoint



Primeiro endpoint criado!!

Case

Todo List

Entrar

E-mail

Senha

Entrar

Todo List

 [Entrada](#)

 [Hoje](#)

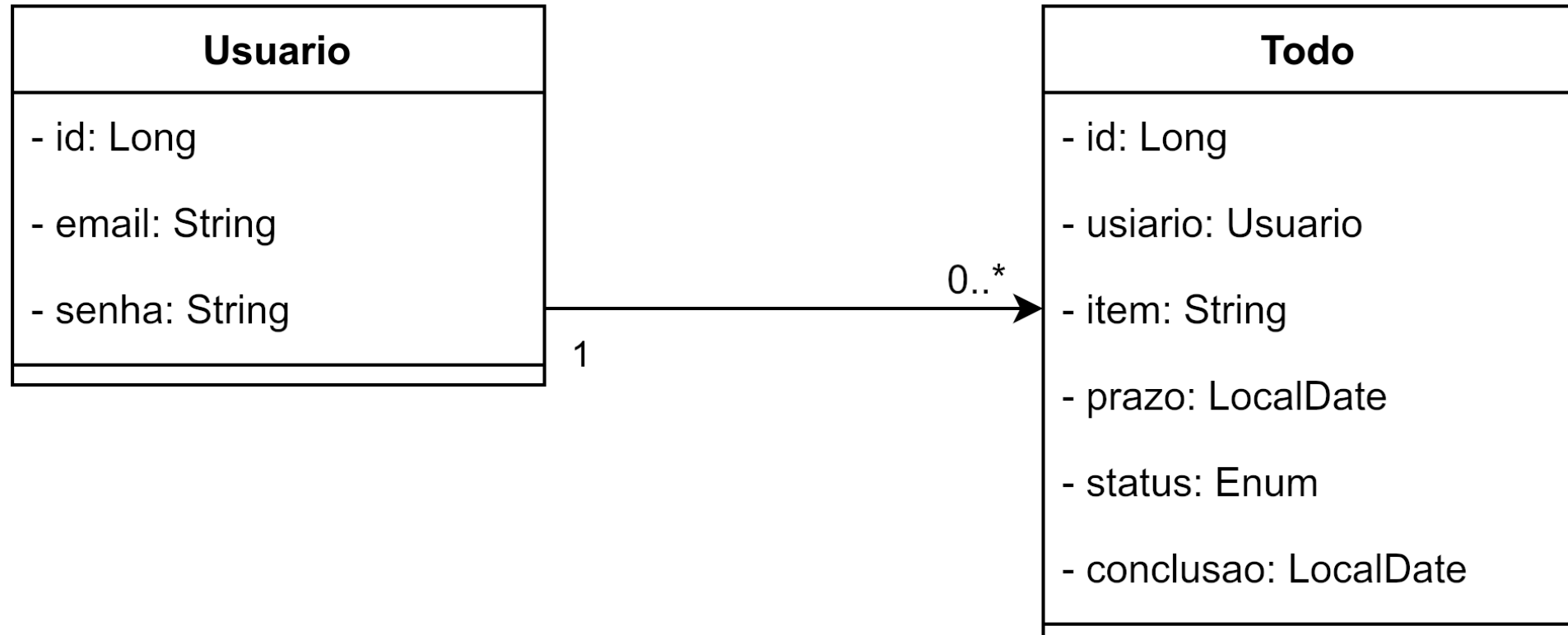
Entrada

- ☐ 2023-07-05 testando
- ☐ 2023-07-05 adicionando outros
- ☐ 2023-05-08 Teste 2

Adicionar tarefa

[sair](#)

Entidades



TodoController

- Como vocês projetariam essa API?
- Quais métodos?
- Quais endpoints?



TodoController

```
@RestController
```

```
@RequestMapping(🌐✓"/api/v1/users/{userId}/todos")
```

```
public class TodoController {
```

```
    @GetMapping()🌐✓
```

```
    public ResponseEntity<List<TodoResponse>> listar(
```

```
        @PathVariable Long userId) {
```

```
        return null;}
```

```
    @GetMapping(🌐✓"/{todoId}")
```

```
    public ResponseEntity<TodoResponse> buscarPorId(
```

```
        @PathVariable Long userId,
```

```
        @PathVariable Long todoId) {
```

```
        return null;}
```

TodoController

@PostMapping

```
public ResponseEntity<TodoResponse> criar(  
    @PathVariable Long userId,  
    @RequestBody TodoCreate dto,  
    UriComponentsBuilder uriBuilder) {  
    return null;}  

```

@PutMapping("/{todoId"})

```
public ResponseEntity<TodoResponse> atualizar(  
    @PathVariable Long userId,  
    @PathVariable Long todoId,  
    @RequestBody TodoUpdate dto) {  
    return null;}  

```

TodoController

```
@PatchMapping(🌐"/{todoId}")
public ResponseEntity<TodoResponse> alterarParcial(
    @PathVariable Long userId,
    @PathVariable Long todoId,
    @RequestBody TodoPatch dto) {
    return null;
}

@DeleteMapping(🌐"/{todoId}")
public ResponseEntity<Void> remover(
    @PathVariable Long userId,
    @PathVariable Long todoId) {
    return null;}
}
```

DTOs

```
public record TodoCreate(String item, LocalDate prazo, Estado estado) {  
    public TodoCreate{ no usages  
        if(item==null || item.isBlank())  
            throw new IllegalArgumentException("O item é obrigatório!");  
        if(prazo==null) prazo = LocalDate.now();  
        if(estados==null) estado=Estado.EM_ANDAMENTO;  
    }  
}
```

```
public enum Estado { 4 usages  
    EM_ANDAMENTO, 2 usages  
    CONCLUIDO, no usages  
    ATRASADO no usages  
}
```


DTOs

```
public record TodoResponse(Long id, String item, LocalDate prazo, Estado estado) { 6 us
    public TodoResponse { no usages
        if(id==null)
            throw new IllegalArgumentException("O id é obrigatório!");
        if(item==null|| item.isBlank())
            throw new IllegalArgumentException("O item é obrigatório!");
        if(prazo==null) throw new IllegalArgumentException("O prazo é obrigatório!");
        if(estados==null) throw new IllegalArgumentException("O estado é obrigatório!");
    }
}
```

```
public record TodoUpdate(String item, LocalDate prazo, Estado estado) { 2 usages
    public TodoUpdate { no usages
        if (item == null || item.isBlank())
            throw new IllegalArgumentException("O item é obrigatório!");
        if (prazo == null) throw new IllegalArgumentException("O Prazo é obrigatório!");
        if (estado == null) throw new IllegalArgumentException("O estado é obrigatório!");
    }
}
```

DTOs

```
public record TodoPatch(String item, LocalDate prazo, Estado estado) {  
  
}
```

DTOs

- E os DTOs do usuário?
- **Implementem**



Criar pacotes no domain

 java

▼  br.edu.ufersa.pw.todo.todoAPI

▼  api

>  controllers

>  dtos

▼  domain

 entities

 repositories

 TodoApiApplication

Criar a classe TodoEntity

Todo

- id: Long
- usuario: Usuario
- ▶ - item: String
- prazo: LocalDate
- status: Enum
- conclusao: LocalDate



Criar a classe UsuarioEntity

Usuario
- id: Long - email: String - senha: String



Configurar o acesso ao BD



application.properties x



TodoController.java



TodoPatch.java



Estado.java



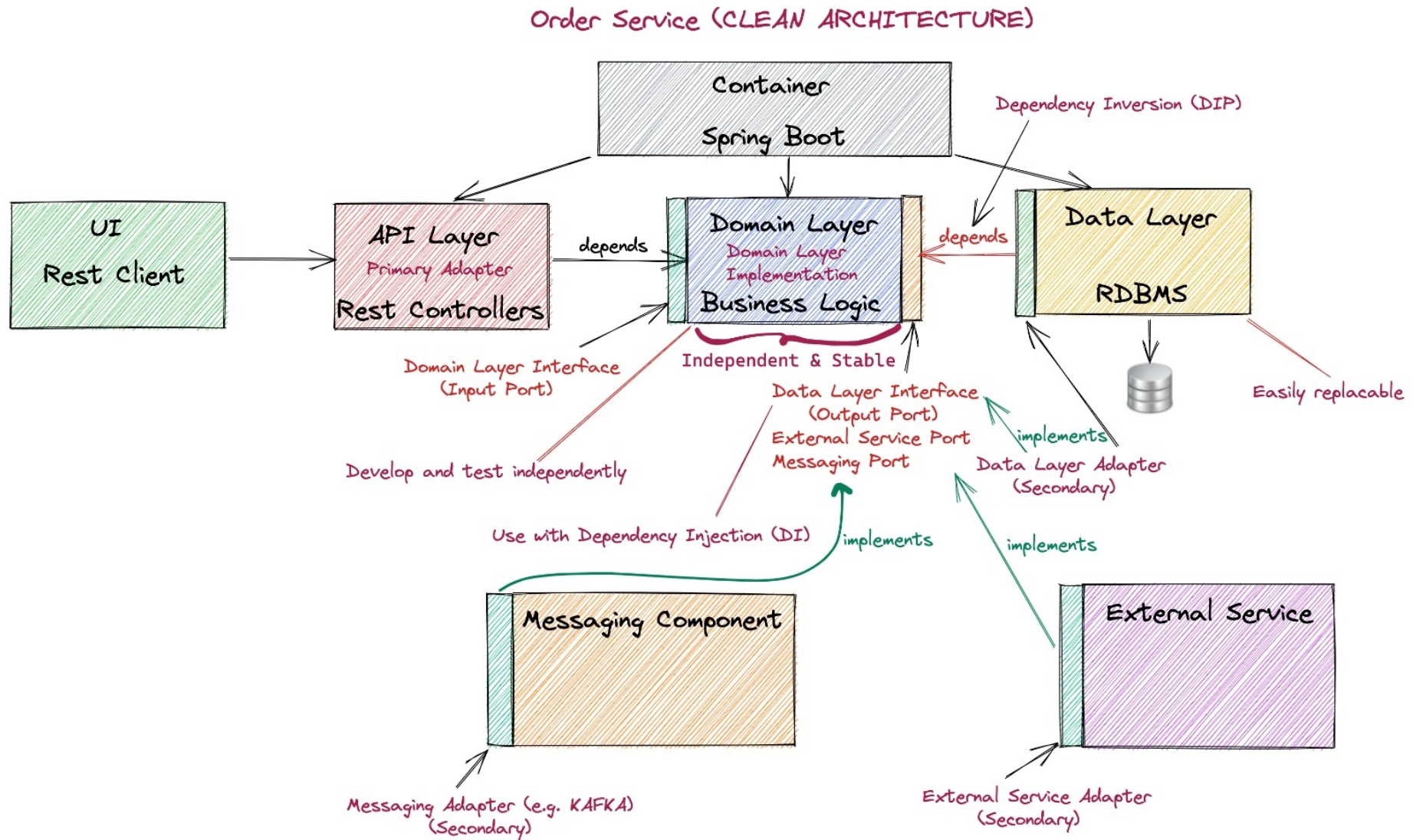
TodoResponse.java



Todo.java

```
spring.application.name=todoAPI
var1=valor definido no app.properties
spring.datasource.url=jdbc:mariadb://localhost:3306/todo?createDatabaseIfNotExist=TRUE
spring.datasource.username=pw
spring.datasource.password=M3lhormateria!
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

Reorganizando o Controller



Próxima Aula

Main Class	@SpringBootApplication	Spring Boot auto configuration
REST Endpoint	@RestController	Class with REST endpoints
	@RequestMapping	REST endpoint method
	@PathVariable	URI path parameter
	@RequestBody	HTTP request body
Periodic Tasks	@Scheduled	Method to run periodically
	@EnableScheduling	Enable Spring's task scheduling
Beans	@Configuration	A class containing Spring beans
	@Bean	Objects to be used by Spring IoC for dependency injection
Spring Managed Components	@Component	A candidate for dependency injection
	@Service	Like @Component
	@Repository	Like @Component, for data base access
Persistence	@Entity	A class which can be stored in the data base via ORM
	@Id	Primary key
	@GeneratedValue	Generation strategy of primary key
	@EnableJpaRepositories	Triggers the search for classes with @Repository annotation
	@EnableTransactionManagement	Enable Spring's DB transaction management through @Beans objects
Miscellaneous	@Autowired	Force dependency injection
	@ConfigurationProperties	Import settings from properties file
Testing	@SpringBootTest	Spring integration test
	@AutoConfigureMockMvc	Configure MockMvc object to test HTTP queries